Click-to-Source 3D

Automatic Instrumentation — Experiment Design

Five Hypotheses for Eliminating Manual Tagging

---

1. Why Now

The project has deferred one decision since Stage 2: how automatic instrumentation should identify which code to instrument. Three candidates were documented — directory-based configuration, comment annotation, and a wrapper API — with weighted selection criteria and no choice made. The stated reason for deferring was that the decision should only be made once real experience existed from tagging real generators by hand.

That experience now exists. Stage 4 tagged four objects across two rendering patterns and produced five distinct defects, all traceable to one cause:

Defect	Cause
Metadata drift	`args` hand-typed as `-26`; code had moved to `-30`
Naming mismatch	display key `waterLevel` vs declared `WATER_LEVEL`
Wrong line number	tag said 176, block was at 178
Dead argument	`terrainSize` is a prop; no literal exists to edit
Source contorted for metadata	three constants merged onto one line so one `line` could address them

Every one is structurally impossible under emission, because a transform reads values from the AST rather than from a developer's memory. That is the argument for automation, and it is now evidence-backed rather than anticipated.

But Stage 4 also produced a counter-argument that the original plan did not account for, and it is the reason this document exists rather than a straightforward "build Stage 6.5" instruction. That counter-argument is in Section 3.

---

2. The Problem, Decomposed

"Click a mesh and get the code without manual tagging" is two problems with very different difficulty. Conflating them is why the question has stayed open.

Problem A — Which file, function, and line created this object

Solvable, with precedent. React DevTools already does exactly this for DOM elements: a Babel plugin injects `__source: {fileName, lineNumber}` into every JSX element at build time, with zero developer effort. For any

object created through JSX — `<mesh>`, `<Water>`, `<Terrain>` — the same technique applies directly.

This half is not research. It is engineering with a known answer.

Problem B — Which runtime arguments produced this specific instance

This is the hard half, and it is the project's differentiator. A build-time transform can inject "this came from line 178." It cannot know that "this particular tree" had `x: -135.678, yaw: 4.858`. Those values exist only at runtime, inside a placement loop, produced by a seeded RNG.

Capturing them requires something that executes — an interception point where real values flow through — or a way to recover them after the fact.

All five hypotheses below are attempts at Problem B. Problem A is assumed solved and is not experimented on.

---

3. The Bar for Success, and Why Stage 4 Set It

The concrete target is `Trees.jsx`: 260 instances across five instanced meshes, each carrying `x`, `z`, `height`, `scale`, and `yaw`, currently produced by hand-written code inside the placement loop.

Two things make this the right test case rather than an arbitrary one.

It is the case the original Stage 6.5 design does not reach. Auto-tagging a generator function emits one tag per call. `Trees.jsx` has no `createTree(x, z, seed)` function — the per-tree work is inlined directly in the loop body. A function-level transform would produce nothing useful here. Since per-instance provenance is what distinguishes this tool from Needle Inspector and the Threejs Devtools MCP server, a mechanism that cannot reach loop bodies solves the easy half of the product and misses the half it is differentiated on.

Stage 4 produced ground truth. Four objects are tagged with known-correct provenance, hand-verified in a browser. Any experiment below can be validated by comparing its output against `instanceSourceRefs` values that are already confirmed accurate. This is an unusually strong experimental position — most instrumentation work has no oracle to check against.

Pass condition for any hypothesis: recover, with zero or near-zero manual annotation, the same per-instance values currently stored in `Trees.jsx`'s `instanceSourceRefs`, and verify them against the existing hand-tagged data.

---

4. Hypothesis 1 — Wrapper API

Mechanism

The developer wraps each generator once; the wrapper intercepts calls and records arguments as they execute.

```
const createTree = clickToSource(function createTree(x, z, seed) { ... })
```

A build-time transform injects the source location into the wrapper call, so the wrapper pairs "where this was declared" with "what was passed this time."

What it solves

Both problems, in principle. It is executable, so it observes runtime values — which is why it is the only one of the three documented Stage 2 candidates that can reach Problem B at all. Directory configuration instruments by file location and cannot intercept arguments; comment annotations are not executable.

What it does not solve

It is not zero-effort, and for `Trees.jsx` it is not even low-effort. There is currently no function to wrap. The per-tree body is inlined in the loop. Adopting this would require refactoring the generator to extract that body into a callable function first — which means the mechanism's cost is not "wrap once per generator" but "restructure the generator, then wrap."

That is a materially different ask, and it is the honest reason this is listed as a hypothesis to test rather than assumed as the answer.

Experiment

1. Extract `Trees.jsx`'s per-tree loop body into a standalone function taking the values it currently closes over.

2. Wrap it with a minimal `clickToSource()` implementation that records arguments per call.

3. Compare recorded arguments against the existing `instanceSourceRefs` for the same seed.

Pass: recovered args match ground truth for all 260 instances, and the refactor does not change rendered output.

What a failure tells you: if the extraction proves awkward or changes behaviour, that is direct evidence the wrapper model imposes a real structural tax on generator code — which should weigh heavily against it as the chosen opt-in strategy.

Cost: highest of the five. Requires real refactoring of a working generator.

---

5. Hypothesis 2 — Call-Site Rewriting

Mechanism

A Babel transform rewrites call sites rather than function declarations:

```
createTree(x, z, seed)
        ↓
__cts(createTree, {file, line, fn})(x, z, seed)
```

The compiler performs the wrapping. The developer writes nothing.

Why this matters more than it first appears

The earlier analysis concluded that only the wrapper API could capture runtime arguments, and therefore that the Stage 2 decision was effectively already made. **That conclusion assumed the transform tags declarations.** If it tags call sites instead, directory-based opt-in gains full argument capture — which would reopen a decision believed closed, and with the lowest developer burden of any option.

Confirming or refuting this is the single most decision-relevant experiment in this document.

What it does not solve

It requires a call to rewrite. `Trees.jsx`'s loop contains `generator.sample(jx, jz)` and `dummy.position.set(...)`, but no single call whose arguments constitute the tree's provenance. Rewriting every call site would capture `sample()`'s two arguments — not the composite per-instance state that actually describes the tree.

So this may solve Problem B for single-object generators (Terrain, Water) and still miss instanced ones.

Experiment

1. Write a minimal Babel plugin that rewrites exactly one call expression in `Terrain.jsx`.

2. Confirm the captured arguments are correct at runtime.

3. Then attempt the same against `Trees.jsx` and determine what, if anything, it captures per instance.

Pass, part 1: the transform captures correct arguments for a single-object generator without breaking the build.

Pass, part 2: it captures something usefully per-instance in the loop. Part 2 failing while part 1 passes is itself a clear, useful result.

What a failure tells you: if call-site rewriting cannot express per-instance provenance, the Stage 2 decision genuinely does narrow to the wrapper API, and that should be recorded as settled.

Cost: roughly half a day. Requires build tooling but only a minimal plugin.

---

6. Hypothesis 3 — Deterministic Replay

Mechanism

Do not record provenance. Recompute it.

Generation is seeded and deterministic — `mulberry32(seed)` throughout. So instead of storing 260 provenance records, store three facts: the seed, the generator identity, and the instance index. At click time, re-run the generator deterministically up to instance N and recover its arguments.

Why this is the most interesting of the five

Drift becomes structurally impossible, not merely unlikely. Every Stage 4 defect was a divergence between recorded metadata and actual code. Replay has no recorded metadata to diverge — the values are recomputed by the same code that produced them. The entire defect class disappears rather than being mitigated.

Memory cost goes to zero. Per-instance storage currently scales linearly with scene size. Replay stores a constant three fields per generator regardless of instance count. On the two-core, 2 GB-VRAM hardware this project targets, that is not a minor consideration.

It is genuinely different from how existing tools approach this. Needle Inspector and the Threejs Devtools MCP server both inspect live runtime state. "Recompute provenance on demand from a seed" is a different answer to the same question, and it is the one idea in this document that could be described as novel rather than as an engineering choice among known options.

What it does not solve

It requires determinism and re-executability. Generation must be free of side effects, independent of external mutable state, and callable in isolation. This holds in the current project but would not hold universally — which makes it a powerful special case rather than a general mechanism.

Replay also has a click-time cost. Re-running a placement loop to reach instance 4,213 is fast, but not free.

Experiment

No build tooling required at all. Write a script that:

1. Takes seed 42 and a target instance index.

2. Re-runs `Trees.jsx`'s placement loop in isolation.

3. Compares the recovered `x`, `z`, `height`, `scale`, `yaw` against the stored `instanceSourceRefs` for that index.

Pass: recovered values match ground truth exactly, for a sample of indices spanning early, middle, and late instances.

What a failure tells you: if values diverge, generation is not as deterministic as assumed — which is itself important to know, because several other design assumptions in this project rest on it.

Cost: a few hours. Cheapest experiment here, needs no transform, no plugin, and no changes to the running app.

---

7. Hypothesis 4 — Execution-Context Frames

Mechanism

Maintain a shadow stack. Instrumented call sites push a provenance frame on entry and pop on exit; anything constructed or written while a frame is live is attributed to that frame.

Pair this with intercepting `InstancedMesh.prototype.setMatrixAt`, and the loop iteration that produced instance i becomes recoverable.

What it solves that nothing else does

The correlation problem. A loop pushes matrices into an array; later, `setMatrixAt(i, m)` writes them. Nothing currently links iteration to index — which is precisely why `Trees.jsx` has to build parallel

`instanceSourceRefs` arrays by hand and keep their ordering in sync. Frames would establish that link automatically.

For synchronous loops — which is what all generation here is — a plain array-as-stack suffices. No async context tracking is needed.

What it does not solve

Something still has to establish the frames, so this composes with another mechanism rather than replacing it. Monkey-patching a Three.js prototype method is also invasive and could interact badly with other libraries doing the same.

Experiment

1. Patch `setMatrixAt` in the consuming project to log its index argument.

2. Maintain a manual frame stack in `Trees.jsx`'s loop — push before, pop after.

3. Log `(index, frame)` pairs and confirm the pairing is correct across all five instanced meshes, including the trunk mesh whose indices span every colour group.

Pass: every logged index maps to the correct loop iteration, verified against ground truth.

What a failure tells you: if pairing breaks — particularly across the trunk mesh, which aggregates instances from all four foliage groups — the correlation problem is harder than a simple stack, and any automated instanced tagging will need a more sophisticated mechanism.

Cost: a few hours. No build tooling.

---

8. Hypothesis 5 — Dummy Proxying

Mechanism

Virtually all R3F instancing code follows one idiom: a shared `dummy = new THREE.Object3D()`, set position/rotation/scale, call `updateMatrix()`, clone the matrix. Proxy that object, record its state at each `updateMatrix()` call, and pair the nth recording with the nth matrix pushed.

What it solves

Per-instance capture at near-zero cost, with no build tooling and no developer annotation — provided the code follows the idiom.

What it does not solve

It is pattern-specific by construction. Code that transforms matrices directly, uses multiple dummies, or writes to the instance buffer out of order would break it silently. Silent breakage is the worst failure mode for a provenance tool, since the output looks plausible.

Its saving grace is that the pattern is close to universal — and this project has four real generators to test that claim against rather than assume it.

Experiment

1. Wrap `Trees.jsx`'s `dummy` in a `Proxy` that records position, rotation, and scale on each `updateMatrix()` call.

2. Compare the recorded sequence against `instanceSourceRefs`, in order.

3. Repeat against `GroundCover.jsx`, which uses the same idiom with a different loop structure and five meshes.

Pass: recorded sequence matches ground truth in both files, in order, with no drift across the full instance count.

What a failure tells you: if it works on `Trees.jsx` but not `GroundCover.jsx`, the idiom is less universal than assumed and this approach is unsuitable as a general mechanism — a useful negative result obtained cheaply.

Cost: a few hours. No build tooling.

---

9. These Are Not Mutually Exclusive

The five are framed as alternatives for clarity, but the likely real answer composes several:

- **H2 (call-site rewriting)** handles Problem A and Problem B for single-object generators.
- **H4 (execution frames)** supplies the loop-iteration-to-instance-index correlation that H2 cannot.
- **H3 (deterministic replay)** could replace per-instance storage entirely, independent of how capture works.
- **H5 (dummy proxying)** is a cheap shortcut that may make H4 unnecessary for the common case.
- **H1 (wrapper API)** is the incumbent and the baseline the others must beat on developer burden.

A plausible end state is H2 for location and simple args, H4 or H5 for instanced correlation, and H3 as a storage optimisation where determinism holds.

---

10. Recommended Sequence

Ordered by information gained per hour spent, following the Stage 1 discipline: cheap experiments first, designed to fail fast, throwaway by construction.

#	Experiment	Cost	Why this order
1	H3 — Deterministic replay	Hours	Cheapest, no tooling, highest novelty; also validates a determinism assumption several other designs depend on

2	H5 — Dummy proxying	Hours	No tooling; if it works across both instanced generators it may make H4 unnecessary
3	H4 — Execution frames	Hours	No tooling; directly attacks the correlation problem that hand-written parallel arrays currently solve
4	H2 — Call-site rewriting	~Half a day	First experiment needing build tooling; most decision-relevant for the deferred Stage 2 choice
5	H1 — Wrapper API	Highest	Requires refactoring a working generator; run last, as the baseline the others are measured against

Run H3 first. It needs no transform, no plugin, and no changes to the running application, and a positive result would be the most interesting outcome available here.

---

11. What None of These Solve

Worth stating so the scope of the eventual answer is not overestimated:

- **Cross-file provenance.** `ALPINE_TERRAIN.worldSize` feeds Terrain, Trees, GroundCover, and Water. Nothing here surfaces that editing it has scene-wide blast radius — the problem that caused `terrainSize` to be dropped from Water's args.
- **Shader values.** GLSL constants remain outside the Babel editor's reach under every hypothesis. This is an explicit v0.1 scope exclusion, not an oversight.
- **Which generation pass created an object.** Meaningful only once generation is explicitly staged, which it currently is not.
- **Instance-level visual identification.** The mesh-wide selection highlight is a prerequisite for rich per-instance data being usable — showing accurate provenance for one tree is undercut if the outline highlights all of them.

---

12. Experimental Discipline

These are Stage 1-style experiments and should be run under Stage 1 rules, which this project has already validated once:

- **Throwaway by construction.** Nothing here is production code. If an experiment starts feeling like an implementation, it has stopped being an experiment.
- **Explicit pass/fail before running.** Each hypothesis above states its criterion. Decide nothing after the fact.

- **A negative result is a successful experiment.** Learning that dummy proxying breaks on `GroundCover.jsx`, or that call-site rewriting cannot reach loop bodies, is worth the hours spent and narrows the decision.
- **Observe, do not fix.** Stage 1's rule applies: when something fails, record it and move on. Do not spend three hours designing a workaround mid-experiment.
- **Ground truth is the oracle.** Every claim gets checked against the hand-verified `instanceSourceRefs` from Stage 4, not against expectation.

---

13. Relationship to the Staged Plan

This work informs the Stage 6.5 decision but does not commit to a schedule for it. Two structural points from the plan review remain open and are affected by whatever these experiments find:

The verification/emission split. A verification pass — checking that existing manual tags still match their source — is a distinct, much cheaper capability than emission, requires no opt-in decision at all, and would have caught four of the five Stage 4 defect classes. It reuses `collectCandidates`, which already walks the AST matching candidates by name and line. Whether to pull that forward ahead of emission is a decision worth making independently of these experiments.

Emission remains the highest-risk component in the project. Nothing here changes that. The purpose of these five experiments is to replace a deferred guess with evidence before that risk is taken — not to skip it.

---

14. Summary

The question "how do we click a mesh and get the code without manual tagging" has been treated as one problem with one deferred answer. It is two problems: source location, which has a known solution and precedent, and per-instance runtime arguments, which is genuinely unsolved and is what the tool is differentiated on.

Five mechanisms could address the second. One is the incumbent. One would reopen a decision believed settled. One would make an entire class of defect structurally impossible. Two are cheap enough to test in an afternoon each.

All five are testable against ground truth that Stage 4 already produced, which is the strongest reason to run them now rather than continue deferring.